

ЗАДАНИЕ 3

Придумайте и решите задачу выбора из 5 вариантов одного наилучшего. Варианты альтернатив оцениваются по 4 критериям, сравнительная важность которых определяется Вами. Все критерии не должны быть равноценны, а варианты альтернатив не должны быть сравнимы по Парето.

Решение

Задача: Выбор страны для отпуска

Альтернативы (страны для отпуска):

1. Италия
2. Таиланд
3. ОАЭ
4. Япония
5. Мексика

Критерии оценки:

1. Стоимость отдыха (тыс. руб., минимизация)
2. Культурное богатство (баллы 1-10, максимизация)
3. Климат и погода (баллы 1-10, максимизация)
4. Уровень безопасности (баллы 1-10, максимизация)

Данные по альтернативам

```
import numpy as np
import pandas as pd

# Данные по альтернативам
countries = ['Италия', 'Таиланд', 'ОАЭ', 'Япония', 'Мексика']
criteria = ['Стоимость', 'Культура', 'Климат', 'Безопасность']

# Матрица оценок (строка - страна, столбец - критерий)
data = np.array([
    [120, 9, 8, 8], # Италия
    [80, 7, 9, 7], # Таиланд
    [150, 6, 7, 9], # ОАЭ
    [130, 10, 7, 10], # Япония
    [90, 8, 10, 6] # Мексика
])
```

```
df = pd.DataFrame(data, index=countries, columns=criteria)
print("ИСХОДНЫЕ ДАННЫЕ:")
print(df)
print()
```

ИСХОДНЫЕ ДАННЫЕ:

	Стоимость	Культура	Климат	Безопасность
Италия	120	9	8	8
Таиланд	80	7	9	7
ОАЭ	150	6	7	9
Япония	130	10	7	10
Мексика	90	8	10	6

Анализ по Парето

```
# АНАЛИЗ ПО ПАРЕТО
print("=" * 60)
print("АНАЛИЗ ПО ПАРЕТО")
print("=" * 60)

def is_pareto_dominant(a, b, minimize_cols=[0]):
    """
    Проверяет, доминирует ли альтернатива а над b по Парето
    Возвращает: 1 если a > b, -1 если b > a, 0 если несравнимы
    """
    a_better = False
    b_better = False

    for i in range(len(a)):
        if i in minimize_cols:
            # Для критериев минимизации (стоимость)
            if a[i] < b[i]:
                a_better = True
            elif a[i] > b[i]:
                b_better = True
        else:
            # Для критериев максимизации
            if a[i] > b[i]:
                a_better = True
            elif a[i] < b[i]:
                b_better = True

    if a_better and not b_better:
        return 1    # a доминирует над b
    elif b_better and not a_better:
        return -1   # b доминирует над a
    else:
        return 0    # несравнимы

# Строим матрицу доминирования по Парето
```

```

pareto_matrix = np.zeros((len(countries), len(countries)), dtype=int)

print("Матрица доминирования по Парето:")
print("    " + " ".join(f"{c:8}" for c in countries))
for i in range(len(countries)):
    row = f"{countries[i]:8}"
    for j in range(len(countries)):
        if i == j:
            pareto_matrix[i, j] = 0
            row += "    -    "
        else:
            dominance = is_pareto_dominant(data[i], data[j])
            pareto_matrix[i, j] = dominance
            if dominance == 1:
                row += "    >    "
            elif dominance == -1:
                row += "    <    "
            else:
                row += "    ~    "
    print(row)

# Находим парето-оптимальные альтернативы
print("\nАНАЛИЗ ПАРЕТО-ОПТИМАЛЬНЫХ АЛЬТЕРНАТИВ:")
pareto_optimal = []

for i in range(len(countries)):
    is_optimal = True
    for j in range(len(countries)):
        if i != j and pareto_matrix[i, j] == -1: # Если кто-то доминирует
над i
            is_optimal = False
            break
    if is_optimal:
        pareto_optimal.append(countries[i])

print(f"Парето-оптимальные альтернативы: {pareto_optimal}")

# Анализ сравнимости
print("\nАНАЛИЗ СРАВНИМОСТИ ПО ПАРЕТО:")
comparable_pairs = []
non_comparable_pairs = []

for i in range(len(countries)):
    for j in range(i + 1, len(countries)):
        if pareto_matrix[i, j] != 0:
            comparable_pairs.append((countries[i], countries[j],
pareto_matrix[i, j]))
        else:
            non_comparable_pairs.append((countries[i], countries[j]))

print(f"Сравнимые пары ({len(comparable_pairs)}):")

```

```

for pair in comparable_pairs:
    symbol = ">" if pair[2] == 1 else "<"
    print(f" {pair[0]} {symbol} {pair[1]}")

print(f"\nНесравнимые пары ({len(non_comparable_pairs)}):")
for pair in non_comparable_pairs:
    print(f" {pair[0]} ~ {pair[1]}")

# Проверяем условие задачи (не должны быть все сравнимы по Парето)
if len(non_comparable_pairs) > 0:
    print(f"\n✓ УСЛОВИЕ ВЫПОЛНЕНО: существуют несравнимые по Парето альтернативы")
else:
    print(f"\nX УСЛОВИЕ НЕ ВЫПОЛНЕНО: все альтернативы сравнимы по Парето")

# Продолжаем с методами выбора...
print("\n" + "=" * 60)
print("МЕТОДЫ ВЫБОРА НАИЛУЧШЕЙ АЛЬТЕРНАТИВЫ")
print("=" * 60)

```

=====

АНАЛИЗ ПО ПАРЕТО

=====

Матрица доминирования по Парето:

	Италия	Таиланд	ОАЭ	Япония	Мексика
Италия	-	~	~	~	~
Таиланд	~	-	~	~	~
ОАЭ	~	~	-	<	~
Япония	~	~	>	-	~
Мексика	~	~	~	~	-

АНАЛИЗ ПАРЕТО-ОПТИМАЛЬНЫХ АЛЬТЕРНАТИВ:

Парето-оптимальные альтернативы: ['Италия', 'Таиланд', 'Япония', 'Мексика']

АНАЛИЗ СРАВНИМОСТИ ПО ПАРЕТО:

Сравнимые пары (1):

ОАЭ < Япония

Несравнимые пары (9):

Италия ~ Таиланд

Италия ~ ОАЭ

Италия ~ Япония

Италия ~ Мексика

Таиланд ~ ОАЭ

Таиланд ~ Япония

Таиланд ~ Мексика

ОАЭ ~ Мексика

Япония ~ Мексика

✓ УСЛОВИЕ ВЫПОЛНЕНО: существуют несравнимые по Парето альтернативы

Нормализация данных:

```
def correct_normalize(matrix, minimize_cols=[0]):
    """Нормализация данных"""
    normalized = np.zeros_like(matrix, dtype=float)

    for j in range(matrix.shape[1]):
        col = matrix[:, j]
        if j in minimize_cols:
            # Для критериев минимизации (стоимость)
            normalized[:, j] = (col.max() - col) / (col.max() - col.min())
        else:
            # Для критериев максимизации
            normalized[:, j] = (col - col.min()) / (col.max() - col.min())

    return normalized

# Нормализуем данные
normalized_data = correct_normalize(data)
df_normalized = pd.DataFrame(normalized_data, index=countries,
                              columns=criteria)

print("НОРМАЛИЗОВАННЫЕ ДАННЫЕ (0-1):")
print(df_normalized.round(3))
print()

# Веса критериев (не равноценны!)
weights = np.array([0.35, 0.25, 0.20, 0.20])

print("ВЕСА КРИТЕРИЕВ:")
for i, criterion in enumerate(criteria):
    print(f"{criterion}: {weights[i]:.0%}")

# Метод взвешенной суммы
weighted_scores = np.sum(normalized_data * weights, axis=1)

print("\nМЕТОД ВЗВЕШЕННОЙ СУММЫ - РЕЗУЛЬТАТЫ:")
results_weighted = pd.DataFrame({
    'Страна': countries,
    'Взвешенная_сумма': weighted_scores
}).sort_values('Взвешенная_сумма', ascending=False)

results_weighted['Ранг'] = range(1, len(countries) + 1)
print(results_weighted.to_string(index=False))
```

НОРМАЛИЗОВАННЫЕ ДАННЫЕ (0-1):

	Стоимость	Культура	Климат	Безопасность
Италия	0.429	0.75	0.333	0.50
Таиланд	1.000	0.25	0.667	0.25
ОАЭ	0.000	0.00	0.000	0.75
Япония	0.286	1.00	0.000	1.00
Мексика	0.857	0.50	1.000	0.00

ВЕСА КРИТЕРИЕВ:
Стоимость: 35%
Культура: 25%
Климат: 20%
Безопасность: 20%

МЕТОД ВЗВЕШЕННОЙ СУММЫ - РЕЗУЛЬТАТЫ:

Страна	Взвешенная_сумма	Ранг
Мексика	0.625000	1
Таиланд	0.595833	2
Япония	0.550000	3
Италия	0.504167	4
ОАЭ	0.150000	5

Метод Подиновского

```
def correct_podinovsky_method(matrix, importance_order,
                                minimize_cols=[0]):
    """Метод Подиновского"""
    working_matrix = matrix.copy().astype(float)

    for col in minimize_cols:
        working_matrix[:, col] = -working_matrix[:, col]

    for j in range(working_matrix.shape[1]):
        col = working_matrix[:, j]
        working_matrix[:, j] = (col - col.min()) / (col.max() - col.min())

    scores = np.zeros(len(matrix))

    for i in range(len(matrix)):
        score = 0
        for j, crit_idx in enumerate(importance_order):
            weight = len(importance_order) - j
            score += working_matrix[i, crit_idx] * weight
        scores[i] = score

    return scores

importance_order = [0, 3, 1, 2] # Стоимость > Безопасность > Культура >
Климат

podinovsky_scores = correct_podinovsky_method(data, importance_order)
results_podinovsky = pd.DataFrame({
    'Страна': countries,
    'Оценка_Подиновского': podinovsky_scores
}).sort_values('Оценка_Подиновского', ascending=False)

results_podinovsky['Ранг'] = range(1, len(countries) + 1)

print(f"\nМЕТОД ПОДИНОВСКОГО - РЕЗУЛЬТАТЫ:")
```

```
print(f"Порядок важности: {[criteria[i] for i in importance_order]}")
print(results_podinovsky.to_string(index=False))
```

МЕТОД ПОДИНОВСКОГО - РЕЗУЛЬТАТЫ:

```
Порядок важности: ['Стоимость', 'Безопасность', 'Культура', 'Климат']
Страна  Оценка_Подиновского  Ранг
Япония      6.142857      1
Таиланд     5.916667      2
Мексика     5.428571      3
Италия      5.047619      4
ОАЭ         2.250000      5
```

Комплексный анализ:

```
def comprehensive_analysis(weighted_results, podinovsky_results,
                             pareto_optimal,
                             weighted_scores, podinovsky_scores, data,
                             countries, criteria):
    """Комплексный анализ с подробным выводом"""
    print("ПОДРОБНЫЙ АНАЛИЗ РЕЗУЛЬТАТОВ")
    print("=" * 70)

    # Собираем все данные для анализа
    final_scores = {}
    print("ДЕТАЛИЗАЦИЯ РЕЗУЛЬТАТОВ ПО КАЖДОЙ АЛЬТЕРНАТИВЕ:")
    print("-" * 70)

    for country in countries:
        idx = countries.index(country)
        weight_rank = weighted_results[weighted_results['Страна'] ==
country]['Ранг'].values[0]
        pod_rank = podinovsky_results[podinovsky_results['Страна'] ==
country]['Ранг'].values[0]
        is_pareto_opt = country in pareto_optimal

        final_scores[country] = {
            'sum_ranks': weight_rank + pod_rank,
            'weighted_rank': weight_rank,
            'podinovsky_rank': pod_rank,
            'weighted_score': weighted_scores[idx],
            'podinovsky_score': podinovsky_scores[idx],
            'is_pareto_optimal': is_pareto_opt,
            'data': data[idx]
        }

    # Подробный вывод по каждой стране
    print(f"\n{country.upper():10}")
```

```

        print(f"    Метод взвешенной суммы: {weighted_scores[idx]:.3f}
(ранг: {weight_rank})")
        print(f"    Метод Подиновского:      {podinovsky_scores[idx]:.3f}
(ранг: {pod_rank})")
        print(f"    Сумма рангов:                  {weight_rank + pod_rank}")
        print(f"    Парето-оптимальность:      {'✓ ДА' if is_pareto_opt else
'X НЕТ'}")

    # Анализ сильных и слабых сторон
    print(f"    Оценки по критериям:")
    for j, criterion in enumerate(criteria):
        value = data[idx, j]
        max_val = data[:, j].max()
        min_val = data[:, j].min()

        if j == 0: # Стоимость
            status = "✓ ЛУЧШИЙ" if value == min_val else "X НЕ
ЛУЧШИЙ"

            print(f"        {criterion}: {value} тыс.руб. {status}")
        else:
            status = "✓ ЛУЧШИЙ" if value == max_val else "X НЕ
ЛУЧШИЙ"

            print(f"        {criterion}: {value}/10 {status}")

    # ФИНАЛЬНАЯ РАНЖИРОВКА
    print("\n" + "=" * 70)
    print("ФИНАЛЬНАЯ РАНЖИРОВКА АЛЬТЕРНАТИВ")
    print("=" * 70)

    final_ranking = sorted(final_scores.items(), key=lambda x:
x[1]['sum_ranks'])

    print("\nРейтинг (по сумме рангов, чем меньше - тем лучше):")
    print("-" * 70)
    print(f"{'Место':<6} {'Страна':<10} {'Сумма рангов':<12}
{'Взвеш.ранг':<10} {'Под.ранг':<8} {'Парето':<8}")
    print("-" * 70)

    for i, (country, scores) in enumerate(final_ranking, 1):
        pareto_status = "✓" if scores['is_pareto_optimal'] else "X"
        print(f"{'i':<6} {country:<10} {scores['sum_ranks']:<12}
{scores['weighted_rank']:<10} {scores['podinovsky_rank']:<8}
{pareto_status:<8}")

    best_country = final_ranking[0][0]
    best_scores = final_scores[best_country]

    # АНАЛИЗ ЛУЧШЕЙ АЛЬТЕРНАТИВЫ
    print("\n" + "=" * 70)
    print(f"АНАЛИЗ ЛУЧШЕЙ АЛЬТЕРНАТИВЫ: {best_country.upper()}")
    print("=" * 70)

```



```

print(f"\nОБЩАЯ ХАРАКТЕРИСТИКА:")
print(f"    • Сумма рангов: {best_scores['sum_ranks']} (лучший
показатель)")
print(f"    • Ранг по взвешенной сумме: {best_scores['weighted_rank']}")
print(f"    • Ранг по методу Подиновского:
{best_scores['podinovsky_rank']}")
print(f"    • Парето-оптимальность: {'✓ ДА - никто не доминирует над
этой альтернативой' if best_scores['is_pareto_optimal'] else 'X НЕТ -
существует доминирующая альтернатива'}")

print(f"\nКЛЮЧЕВЫЕ ПРЕИМУЩЕСТВА {best_country.upper()}:")
advantages = []

# Анализ по критериям
best_idx = countries.index(best_country)

if data[best_idx, 0] == data[:, 0].min():
    advantages.append(f"САМАЯ НИЗКАЯ СТОИМОСТЬ ({data[best_idx, 0]}
тыс.руб.) - экономия бюджета")
elif data[best_idx, 0] <= data[:, 0].mean():
    advantages.append(f"НИЗКАЯ СТОИМОСТЬ ({data[best_idx, 0]}
тыс.руб.) - хорошее соотношение цены")

if data[best_idx, 1] == data[:, 1].max():
    advantages.append(f"ЛУЧШАЯ КУЛЬТУРА ({data[best_idx, 1]}/10) -
богатое культурное наследие")
elif data[best_idx, 1] >= data[:, 1].mean():
    advantages.append(f"ВЫСОКАЯ КУЛЬТУРА ({data[best_idx, 1]}/10) -
интересные достопримечательности")

if data[best_idx, 2] == data[:, 2].max():
    advantages.append(f"ЛУЧШИЙ КЛИМАТ ({data[best_idx, 2]}/10) -
идеальные погодные условия")
elif data[best_idx, 2] >= data[:, 2].mean():
    advantages.append(f"ХОРОШИЙ КЛИМАТ ({data[best_idx, 2]}/10) -
комфортные условия для отдыха")

if data[best_idx, 3] == data[:, 3].max():
    advantages.append(f"ЛУЧШАЯ БЕЗОПАСНОСТЬ ({data[best_idx, 3]}/10) -
максимальная защищенность")
elif data[best_idx, 3] >= data[:, 3].mean():
    advantages.append(f"ВЫСОКАЯ БЕЗОПАСНОСТЬ ({data[best_idx, 3]}/10)
- надежные условия")

if best_scores['weighted_rank'] == 1:
    advantages.append("ЛИДЕР по методу взвешенной суммы - лучший
баланс по всем критериям")

if best_scores['podinovsky_rank'] == 1:

```

```

        advantages.append("ЛИДЕР по методу Подиновского - оптимален с
учетом важности критериев")

    for i, advantage in enumerate(advantages, 1):
        print(f"    {i}. {advantage}")

    print(f"\nКОМПРОМИССЫ И ОГРАНИЧЕНИЯ:")
    compromises = []

    if data[best_idx, 0] > data[:, 0].min():
        min_cost_country = countries[np.argmin(data[:, 0])]
        compromises.append(f"Стоимость выше, чем у {min_cost_country}
({data[best_idx, 0]} vs {data[:, 0].min()} тыс.руб.)")

    if data[best_idx, 1] < data[:, 1].max():
        max_culture_country = countries[np.argmax(data[:, 1])]
        compromises.append(f"Культура ниже, чем у {max_culture_country}
({data[best_idx, 1]} vs {data[:, 1].max()}/10)")

    if data[best_idx, 2] < data[:, 2].max():
        max_climate_country = countries[np.argmax(data[:, 2])]
        compromises.append(f"Климат хуже, чем у {max_climate_country}
({data[best_idx, 2]} vs {data[:, 2].max()}/10)")

    if data[best_idx, 3] < data[:, 3].max():
        max_safety_country = countries[np.argmax(data[:, 3])]
        compromises.append(f"Безопасность ниже, чем у {max_safety_country}
({data[best_idx, 3]} vs {data[:, 3].max()}/10)")

    if compromises:
        for i, compromise in enumerate(compromises, 1):
            print(f"    {i}. {compromise}")
    else:
        print("    ✓ Идеальный вариант - нет существенных компромиссов")

    print(f"\nСПРАВНЕНИЕ С КОНКУРЕНТАМИ:")
    if len(final_ranking) > 1:
        second_country = final_ranking[1][0]
        second_scores = final_scores[second_country]
        diff = best_scores['sum_ranks'] - second_scores['sum_ranks']

        print(f"    • Преимущество над {second_country}: {abs(diff)} пунктов
в сумме рангов")

        # Анализ почему выиграла лучшая страна
        if best_scores['weighted_rank'] < second_scores['weighted_rank']:
            print(f"    • Сильнее по взвешенной сумме:
{best_scores['weighted_rank']} vs {second_scores['weighted_rank']} место")
            if best_scores['podinovsky_rank'] <
second_scores['podinovsky_rank']:

```

```

        print(f"    • Сильнее по методу Подиновского:
{best_scores['podinovsky_rank']} vs {second_scores['podinovsky_rank']}
место")

    print(f"\n РЕКОМЕНДАЦИЯ:")
    print(f"    {best_country.upper()} - оптимальный выбор для отпуска, так
как:")
    print(f"    1. Демонстрирует лучший баланс по всем методам оценки")
    print(f"    2. {'Является парето-оптимальной' if
best_scores['is_pareto_optimal'] else 'Хотя не является парето-
оптимальной, но показывает лучшие результаты по комплексной оценке'}")
    print(f"    3. Учитывает приоритеты критериев (стоимость → безопасность
→ культура → климат)")

    return best_country

# Вызываем улучшенную функцию
print("\n" + "=" * 70)
best_country = comprehensive_analysis(results_weighted,
results_podinovsky, pareto_optimal,
                                     weighted_scores, podinovsky_scores,
data, countries, criteria)

# ДОПОЛНИТЕЛЬНЫЙ АНАЛИЗ
print("\n" + "=" * 70)
print("ДОПОЛНИТЕЛЬНЫЙ АНАЛИЗ ВЫБОРА")
print("=" * 70)

print(f"\n ОКОНЧАТЕЛЬНОЕ РЕШЕНИЕ: {best_country.upper()}")

print(f"\n КРАТКОЕ ОБОСНОВАНИЕ:")
best_idx = countries.index(best_country)
print(f"• Стоимость: {data[best_idx, 0]} тыс.руб. (минимум: {data[:,
0].min()}, максимум: {data[:, 0].max()})")
print(f"• Культура: {data[best_idx, 1]}/10 (максимум: {data[:,
1].max()})")
print(f"• Климат: {data[best_idx, 2]}/10 (максимум: {data[:, 2].max()})")
print(f"• Безопасность: {data[best_idx, 3]}/10 (максимум: {data[:,
3].max()})")

print(f"\n ВЫВОД:")
print(f"Выбор {best_country} обоснован комплексным подходом,
учитывающим:")
print("✓ Относительную важность критериев")
print("✓ Парето-оптимальность")
print("✓ Результаты двух различных методов многокритериальной
оптимизации")
print("✓ Практические компромиссы между стоимостью и качеством отдыха")

```

=====

ПОДРОБНЫЙ АНАЛИЗ РЕЗУЛЬТАТОВ

=====

ДЕТАЛИЗАЦИЯ РЕЗУЛЬТАТОВ ПО КАЖДОЙ АЛЬТЕРНАТИВЕ:

ИТАЛИЯ

Метод взвешенной суммы: 0.504 (ранг: 4)

Метод Подиновского: 5.048 (ранг: 4)

Сумма рангов: 8

Парето-оптимальность: ✓ ДА

Оценки по критериям:

Стоимость: 120 тыс.руб. X НЕ ЛУЧШИЙ

Культура: 9/10 X НЕ ЛУЧШИЙ

Климат: 8/10 X НЕ ЛУЧШИЙ

Безопасность: 8/10 X НЕ ЛУЧШИЙ

ТАИЛАНД

Метод взвешенной суммы: 0.596 (ранг: 2)

Метод Подиновского: 5.917 (ранг: 2)

Сумма рангов: 4

Парето-оптимальность: ✓ ДА

Оценки по критериям:

Стоимость: 80 тыс.руб. ✓ ЛУЧШИЙ

Культура: 7/10 X НЕ ЛУЧШИЙ

Климат: 9/10 X НЕ ЛУЧШИЙ

Безопасность: 7/10 X НЕ ЛУЧШИЙ

ОАЭ

Метод взвешенной суммы: 0.150 (ранг: 5)

Метод Подиновского: 2.250 (ранг: 5)

Сумма рангов: 10

Парето-оптимальность: X НЕТ

Оценки по критериям:

Стоимость: 150 тыс.руб. X НЕ ЛУЧШИЙ

Культура: 6/10 X НЕ ЛУЧШИЙ

Климат: 7/10 X НЕ ЛУЧШИЙ

Безопасность: 9/10 X НЕ ЛУЧШИЙ

ЯПОНИЯ

Метод взвешенной суммы: 0.550 (ранг: 3)

Метод Подиновского: 6.143 (ранг: 1)

Сумма рангов: 4

Парето-оптимальность: ✓ ДА

Оценки по критериям:

Стоимость: 130 тыс.руб. X НЕ ЛУЧШИЙ

Культура: 10/10 ✓ ЛУЧШИЙ

Климат: 7/10 X НЕ ЛУЧШИЙ

Безопасность: 10/10 ✓ ЛУЧШИЙ

МЕКСИКА

Метод взвешенной суммы: 0.625 (ранг: 1)

Метод Подиновского: 5.429 (ранг: 3)

Сумма рангов: 4
Парето-оптимальность: ✓ ДА
Оценки по критериям:
Стоимость: 90 тыс.руб. X НЕ ЛУЧШИЙ
Культура: 8/10 X НЕ ЛУЧШИЙ
Климат: 10/10 ✓ ЛУЧШИЙ
Безопасность: 6/10 X НЕ ЛУЧШИЙ

=====

ФИНАЛЬНАЯ РАНЖИРОВКА АЛЬТЕРНАТИВ

=====

Рейтинг (по сумме рангов, чем меньше - тем лучше):

Место	Страна	Сумма рангов	Взвеш.ранг	Под.ранг	Парето
1	Таиланд	4	2	2	✓
2	Япония	4	3	1	✓
3	Мексика	4	1	3	✓
4	Италия	8	4	4	✓
5	ОАЭ	10	5	5	X

=====

АНАЛИЗ ЛУЧШЕЙ АЛЬТЕРНАТИВЫ: ТАИЛАНД

=====

ОБЩАЯ ХАРАКТЕРИСТИКА:

- Сумма рангов: 4 (лучший показатель)
- Ранг по взвешенной сумме: 2
- Ранг по методу Подиновского: 2
- Парето-оптимальность: ✓ ДА - никто не доминирует над этой альтернативой

КЛЮЧЕВЫЕ ПРЕИМУЩЕСТВА ТАИЛАНД:

1. САМАЯ НИЗКАЯ СТОИМОСТЬ (80 тыс.руб.) - экономия бюджета
2. ХОРОШИЙ КЛИМАТ (9/10) - комфортные условия для отдыха

КОМПРОМИССЫ И ОГРАНИЧЕНИЯ:

1. Культура ниже, чем у Япония (7 vs 10/10)
2. Климат хуже, чем у Мексика (9 vs 10/10)
3. Безопасность ниже, чем у Япония (7 vs 10/10)

СРАВНЕНИЕ С КОНКУРЕНТАМИ:

- Преимущество над Япония: 0 пунктов в сумме рангов
- Сильнее по взвешенной сумме: 2 vs 3 место

РЕКОМЕНДАЦИЯ:

- ТАИЛАНД - оптимальный выбор для отпуска, так как:
1. Демонстрирует лучший баланс по всем методам оценки
 2. Является парето-оптимальной
 3. Учитывает приоритеты критериев (стоимость → безопасность → культура → климат)

=====

ДОПОЛНИТЕЛЬНЫЙ АНАЛИЗ ВЫБОРА

=====

ОКОНЧАТЕЛЬНОЕ РЕШЕНИЕ: ТАИЛАНД

КРАТКОЕ ОБОСНОВАНИЕ:

- Стоимость: 80 тыс.руб. (минимум: 80, максимум: 150)
- Культура: 7/10 (максимум: 10)
- Климат: 9/10 (максимум: 10)
- Безопасность: 7/10 (максимум: 10)

ВЫВОД:

Выбор Таиланд обоснован комплексным подходом, учитывающим:

- ✓ Относительную важность критериев
- ✓ Парето-оптимальность
- ✓ Результаты двух различных методов многокритериальной оптимизации
- ✓ Практические компромиссы между стоимостью и качеством отдыха